Reinforcement Learning for LLMs

Introduction/Logistics/Scheduling

Agenda

1. Introductions
2. Quick lecture (RL basics)
3. Logistics

About Me

- Post-doc under Prof. Alexander Koller

- My research:
  - RL Theory (for LLMs)
  - Tool-use agents
  - (neurosymbolic LMs and constrained generation)

About You

Crash Course in Classical RL

What is Reinforcement Learning?

- **Reinforcement learning:** learning to make (approximately) optimal* sequences of decisions in an environment through trial-and-error

- **Environment:** modeled as a Markov Decision Process (MDP) $\langle \boldsymbol{S}, \boldsymbol{A}, \boldsymbol{P}, \boldsymbol{R}, \gamma \rangle$
  - $\boldsymbol{S}$: set of *states*
  - $\boldsymbol{A}$: set of *actions*
  - $\boldsymbol{P}$: *transition probability* $P(s_k \mid a_i, s_n)$
  - $\boldsymbol{R}$: *reward function* $R(s) \in (-\infty, \infty)$
  - γ: *discount factor* $\gamma \in (0, 1]$

Environment Example: Grid World

- $S = \{s_1, \ldots, s_9\}$
- $A = \{\Leftarrow, \Rightarrow, \Uparrow, \Downarrow\}$
- $R(s_3) = +1$, $R(s_8) = +3$, $R(s_9) = +10$
  - $R(s_5) = -5$
- Here, we have *deterministic* transitions:
  - $P(s_2 \mid \Rightarrow, s_1) = 1.0$
  - $P(s_5 \mid \Downarrow, s_2) = 1.0$
  - etc.

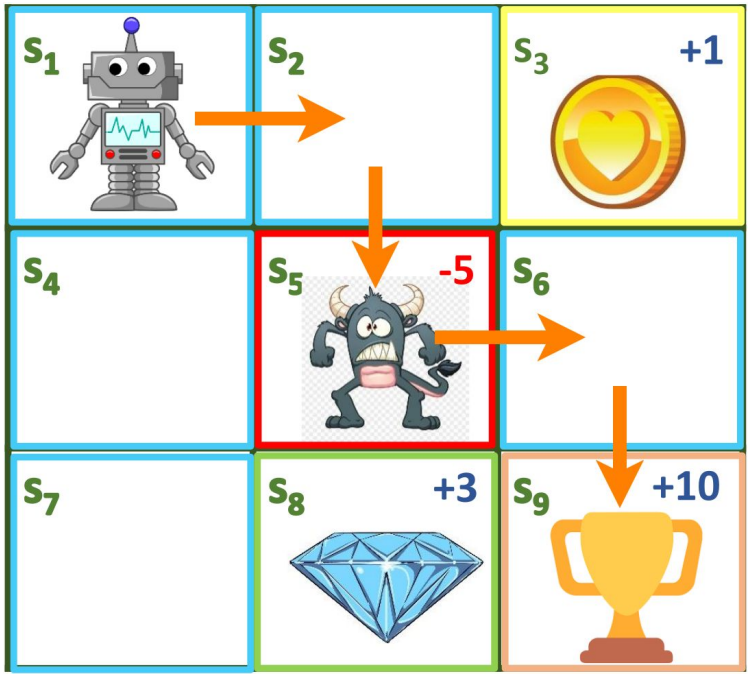


Image: Prof. Dr. Alina Vereshchaka,
CSE 574 Machine Learning, University at Buffalo

Trajectories

- **Trajectory**: sequence of states and actions taken by an agent in an environment (“episode”)
  $\tau = (s_0, a_0, \ldots, a_{n-1}, s_n)$

- Here:
  $\tau = (s_1, \Rightarrow, s_2, \Downarrow, s_5, \Rightarrow, s_6, \Downarrow, s_9)$

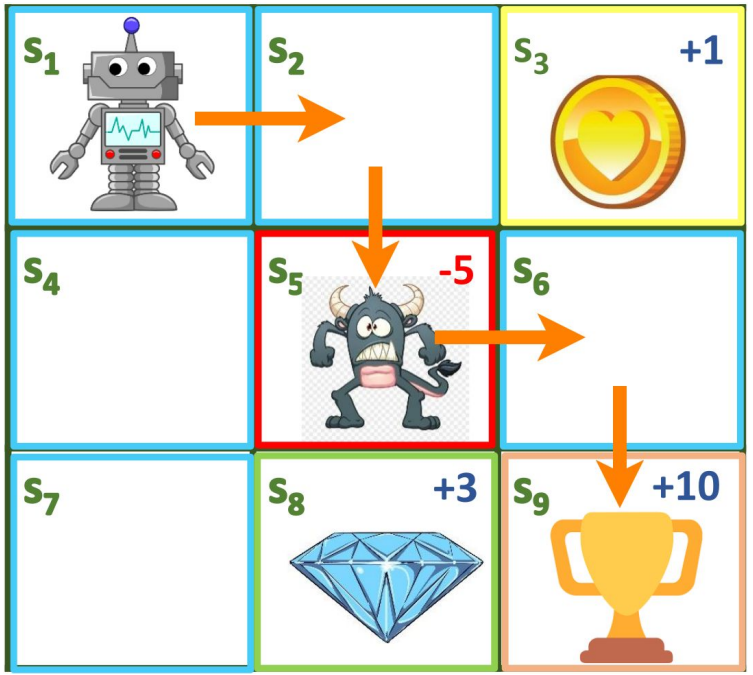


(Optimal) Policies

- **Policy** (π): the agent interacting with the environment
  - $\pi(a \mid s) \in [0, 1]$

- The goal of RL is to learn (approximate) the **optimal policy** π^*
  - π^* ***maximizes expected return*** (reward)

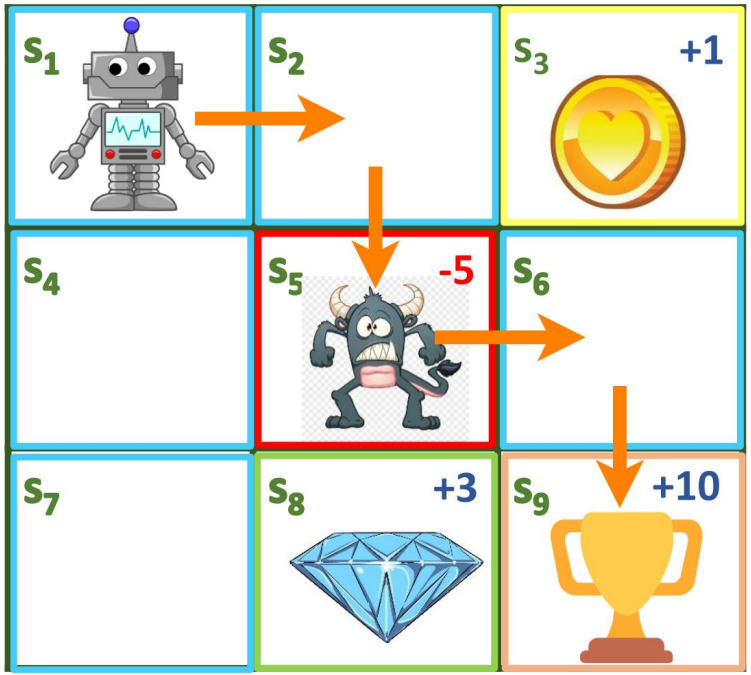
$$\pi^* = \underset{\pi}{\operatorname{argmax}} \underset{\tau \sim \pi}{\mathbb{E}} \left[\sum_{(a_{i-1}, s_i) \in \tau} R(s_i) \right]$$



(Optimal) Policies

❌ *this* trajectory is *not* generated by an optimal policy!

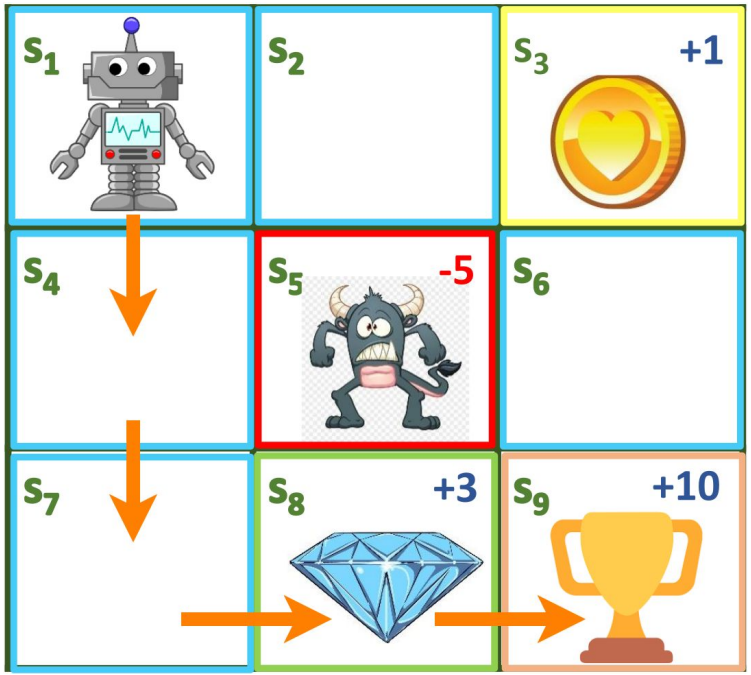
$$\sum_{(a_{i-1},s_i)\in\tau} R(s_i)$$
$$= R(s_2) + R(s_5) + R(s_6) + R(s_9)$$
$$= 0 + (-5) + 0 + 10 = 5$$



(Optimal) Policies

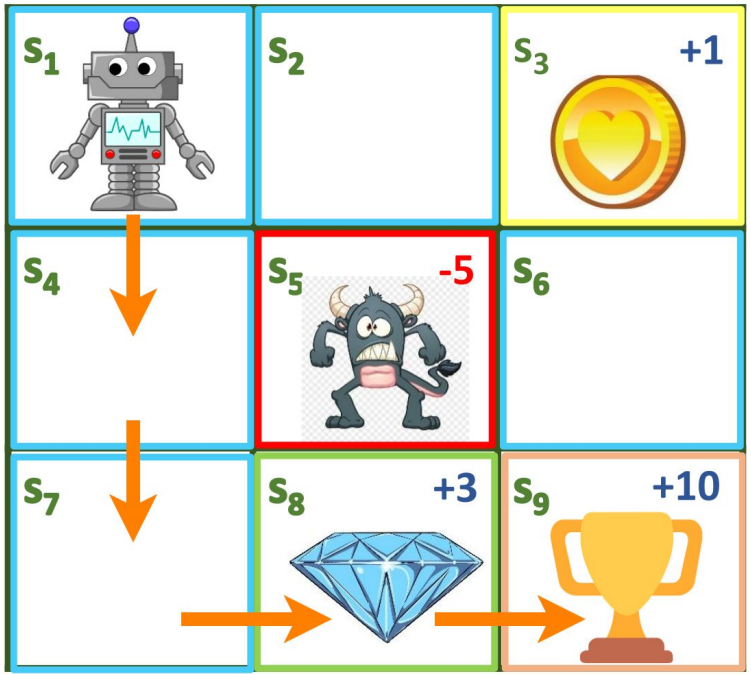
✅ *this* trajectory maximizes expected reward!

$$\sum_{(a_{i-1},s_i)\in\tau} R(s_i)$$
$$= R(s_4) + R(s_7) + R(s_8) + R(s_9)$$
$$= 0 + 0 + 3 + 10 = 13$$



Discount Factor

- … I lied: this trajectory does *not* actually maximize expected reward



Discount Factor

- … I lied: this trajectory does *not* actually maximize expected reward

$\tau = (s_1, \Downarrow, s_4, \Downarrow, s_7, \Rightarrow, s_8, \Rightarrow, s_9)$

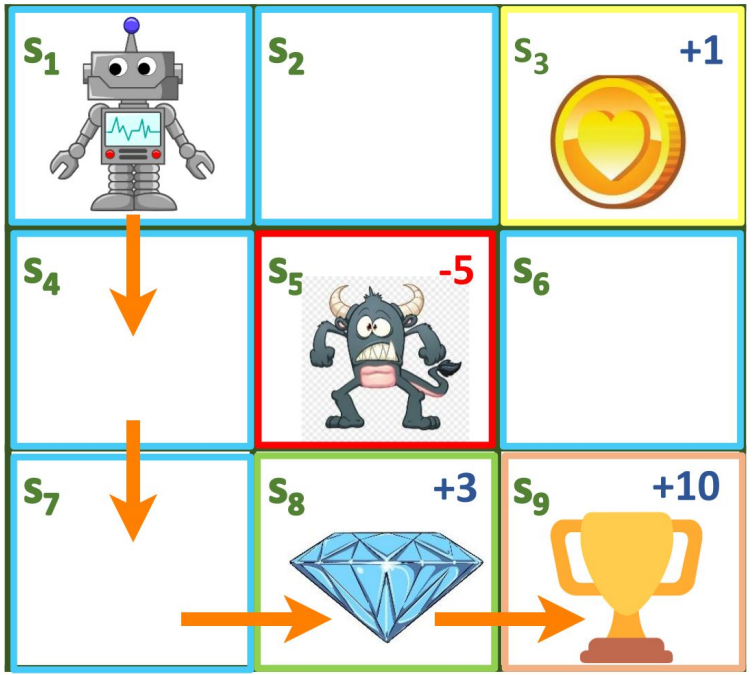
consider:

$\tau' = (s_1, \Downarrow, s_4, \Downarrow, s_7, \Rightarrow, s_8, \Leftarrow, s_7, \Rightarrow, s_8, \Rightarrow, s_9)$

$\tau'' = (\ldots, s_8, \Leftarrow, s_7, \Rightarrow, s_8, \Leftarrow, s_7, \Rightarrow, s_8, \Rightarrow, s_9)$

$\tau''' = \ldots$

etc.

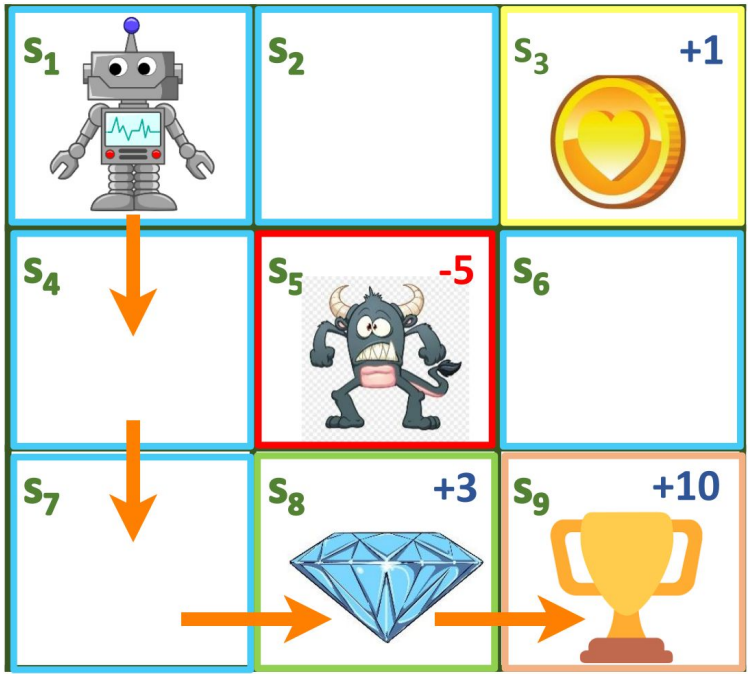


Discount Factor

- Recall: our environment is an MDP $\langle S, A, P, R, \gamma \rangle$, with ***discount factor*** $\gamma \in (0, 1]$

- Maximize expected ~~reward~~ **return**

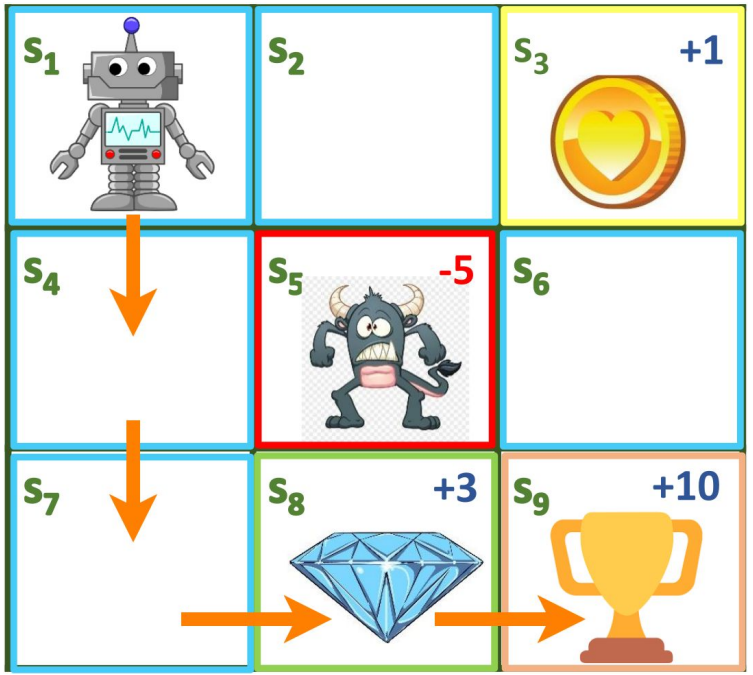
$$\pi^* = \underset{\pi}{\operatorname{argmax}} \underset{\tau \sim \pi}{\mathbb{E}} \left[\sum_{(a_{i-1}, s_i) \in \tau} \gamma^i \cdot R(s_i) \right]$$



Discount Factor

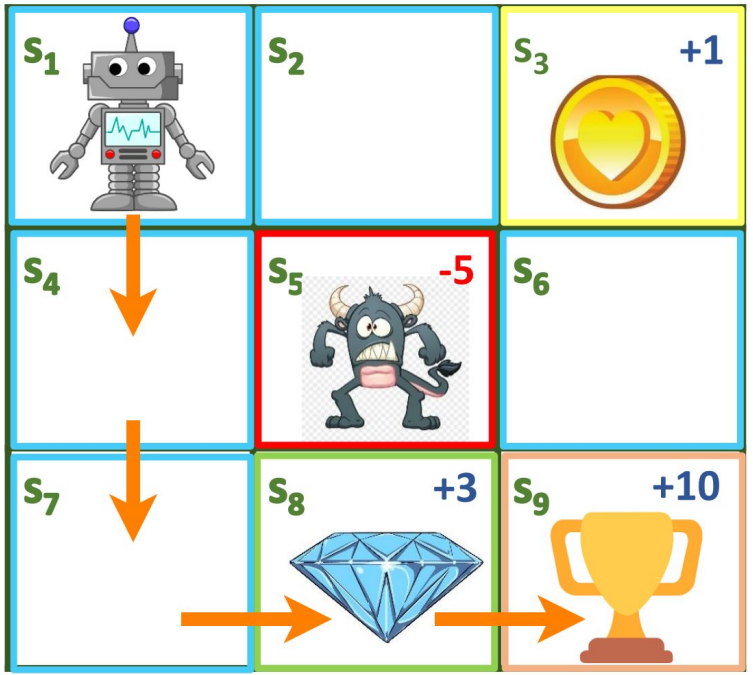
- Recall: our environment is an MDP $\langle S, A, P, R, \gamma \rangle$, with ***discount factor*** $\gamma \in (0, 1]$
- Maximize expected ~~reward~~ **return**

$$\sum_{(a_{i-1}, s_i) \in \tau} \gamma^i \cdot R(s_i)$$
$$= \gamma^0 \cdot R(s_4) + \gamma^1 \cdot R(s_7) + \gamma^2 \cdot R(s_8) + \gamma^3 \cdot R(s_9)$$
$$= 0 + \gamma \cdot 0 + \gamma^2 \cdot 3 + \gamma^3 \cdot 10$$



Discount Factor

- The discount factor *penalizes reward in the future*:
  - Reward at time step *k* is only worth γ^{k-1}
- For example, set γ = 0.9



$$\sum_{(a_{i-1},s_i)\in\tau} 0.9^i \cdot R(s_i)$$
$$= 0.9^0 \cdot R(s_4) + 0.9^1 \cdot R(s_7) + 0.9^2 \cdot R(s_8) + 0.9^3 \cdot R(s_9)$$
$$= 0 + 0 + 0.81 \cdot 3 + 0.729 \cdot 10 = 9.72$$

Value Functions

- **State value function** $V^{\pi}(s)$: expected return under policy π in state s

$$V^{\pi}(s) = \mathop{\mathbb{E}}_{a \sim \pi(- \mid s)} \left[\mathop{\mathbb{E}}_{\hat{s} \sim P(- \mid a, s)} \left[R(s') + \gamma \cdot V^{\pi}(s') \right] \right]$$

- **Action value function** $Q^{\pi}(a, s)$: expected return when taking action a under policy π in state s

$$Q^{\pi}(a, s) = \mathop{\mathbb{E}}_{\hat{s} \sim P(- \mid a, s)} \left[R(s') + \gamma \cdot V^{\pi}(s') \right]$$

Advantage Function

- **Advantage function** $A^{\pi}(a, s)$: expected ***benefit*** from taking action *a* under policy π in state *s*

$$A^{\pi}(a, s) = Q^{\pi}(a, s) - V^{\pi}(s)$$

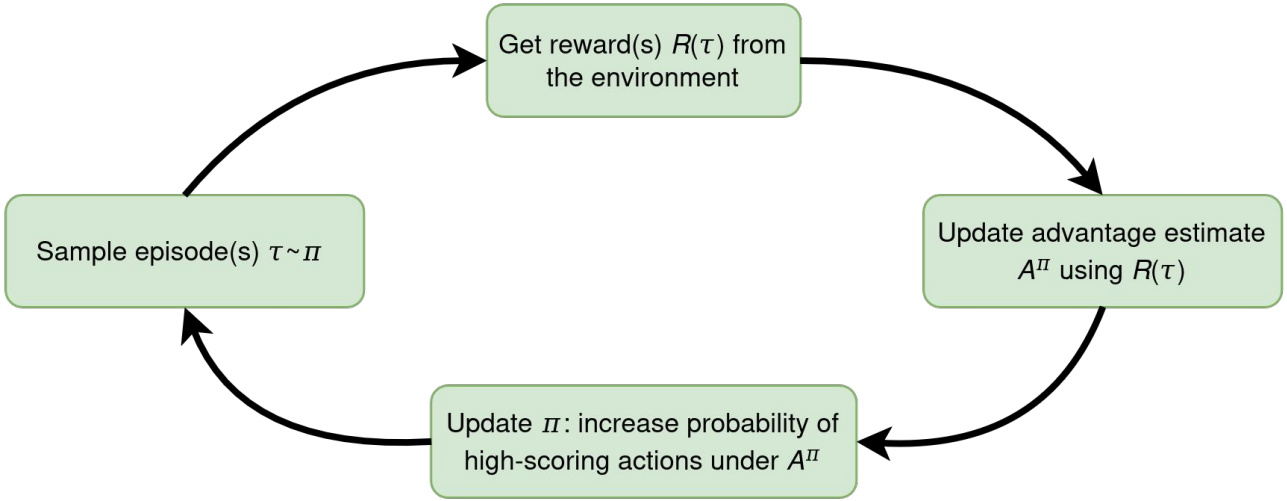
- The optimal action under π in state *s* maximizes $A^{\pi}(a, s)$
  - Equivalently: maximizes $Q^{\pi}(a, s)$

$$a_s^* = \underset{a \in A}{\operatorname{argmax}}\, Q^{\pi}(a, s) = \underset{a \in A}{\operatorname{argmax}}\, A^{\pi}(a, s)$$

- The difference: $A^{\pi}(a, s) > 0$ if *a* is “better than average” (expected)
  - *As long as advantage is positive in training, the policy π is improving*

Online/On-Policy RL Loop

- Outside of toy environments (like our grid world), calculating the advantage function is intractable… it has to be estimated during training!



Exploration vs Exploitation

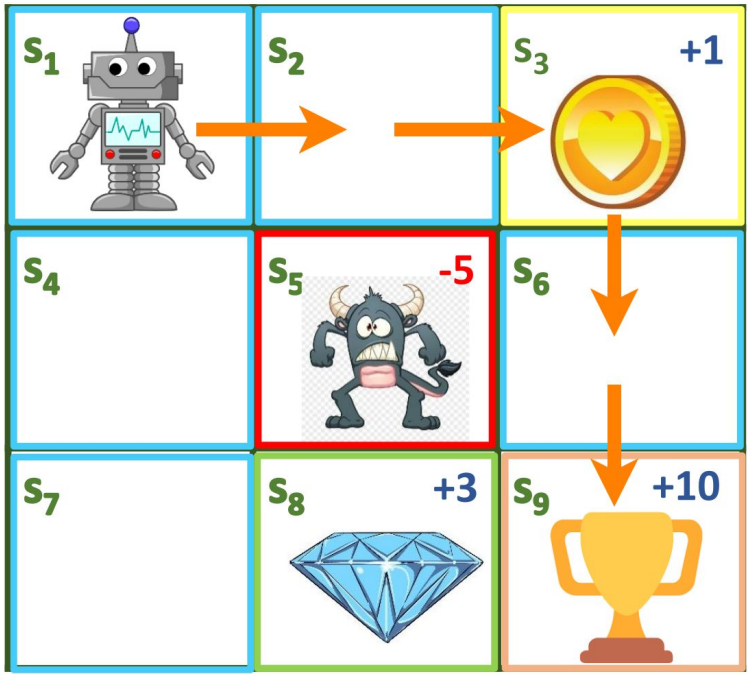
- **Exploration**: initially, we want the agent to take more random actions, to *explore* the environment and get more accurate advantage estimates

- **Exploitation**: once the agent has a feel for which actions are bad, we don't want it to waste time taking them—instead, we want it to concentrate probability onto ("*exploit*") known good actions

Exploration vs Exploitation Tradeoff

- Suppose we take the highlighted path, and we exploit too early
  - Then we'll never discover the lower (optimal) path!

- But if we explore too long, our trajectories can be too noisy to learn from



Sneak Peak:
Adapting RL to LLMs

States and Actions ⇒ Tokens

- Recall that our environment is an MDP $\langle S, A, P, R, \gamma \rangle$
  - States S ⇒ sequences of tokens
  - Actions A ⇒ tokens
  - Transition probability P ⇒ deterministic (concatenation)

- For example, let:
  - s = “Reinforcement learning is”
  - a = “fun!”
- Then:
  - P(“Reinforcement learning is fun!” | a, s) = 1.0

Reward

- Reward is *typically* only assigned to the last state (i.e. the whole sequence), e.g.:
  - RLVR ⇒ $R(\tau)$ = 1 if reasoning trace τ gets the right answer, 0 otherwise
  - RLHF ⇒ $R(\tau)$ assigned by some neural preference model
  - etc.

- Unlike (for example) with a self-driving car, it is difficult to assign (*intrinsic*) intermediate reward to text
  - What makes a reasoning step “good”?
  - How do we know that a half-written program is “heading in the right direction”?

Exploration and Exploitation

- Typically, RL exploration is achieved by randomly sampling actions: this is much more difficult with our LLM setup
  - Language is highly structured, and is easy to irrecoverably break by random sampling
  - It is difficult to cover such a massive action space (hundreds of thousands of tokens)

- As a result, LLMs tend to exploit early on during RL training (**distribution sharpening**)

Scheduling and Logistics

Course Topics/Units

1. **Introduction**:
    a. Concretizing the RL training loop: advantage estimation and policy updating in practice (REINFORCE and PPO)
    b. The latest and greatest LLM RL algorithms (GRPO and its derivatives)

2. **Reward/Credit Assignment**: the importance of reward functions and challenges associated with fine-grained reward computation for LLMs

3. **Training Dynamics**: *what* and *how* LLMs learn during RL training

Schedule/Logistics

https://mjs227.github.io/courses/rl-llms-26